3 KORSZAK / 3 NYELV

A programozási nyelvek evolúciója

Hogyan jutottunk el a számítástechnikától a memóriabiztonságig?

1957	FORTRAN
1972	C
2015	RUST

Miért születnek új nyelvek?

A célok változnak, a korábbi megoldások pedig velünk maradnak.

1957

FORTRAN

Mérnöki számítás

A matematikai feladatot ne gépi utasítássorozatként kelljen leírni.

1972

C

Hordozhatóbb rendszerkód

A Unix fejlesztéséhez kellett hatékony, kifejező rendszernyelv.

2015

RUST 1.0

Memóriabiztonság

A nagy teljesítmény és a fordításidejű biztonság együtt fontos.

FORTRAN: közelebb a képletekhez

IBM / John Backus és csapata / 1957



Tudományos és numerikus számítások

MODERN FORTRAN 2008

```
allocate(scores(5))
scores = [12, 18, 25, 31, 44]
total = sum(scores)
deallocate(scores)
```

Statikus típusok · Tömbműveletek · ALLOCATABLE

C: kontroll közel a hardverhez

Dennis Ritchie / Bell Labs / a korai Unix igényei

C11

```
int *scores = malloc(...);
if (!scores) return 1;

/* feldolgozás */

free(scores);
```



Rust: biztonság a fordítás idején

Ownership + borrowing, hagyományos garbage collector nélkül

`Vec<i32>`
scores

`&scores`
borrow / kölcsönzés

`calculate(&[i32])`

Élettartam- és kölcsönzési szabályok

Sok memóriahiba korábban kiszűrhető

A lényeg egyetlen táblázatban

Minden nyelv más erősségekre optimalizál.

SZEMPONT	FORTRAN	C	RUST
Fő terület	Tudományos	Rendszerkód	Biztonságos rendszerkód
Típusosság	Statikus	Statikus	Statikus, szigorú
Memóriakezelés	ALLOCATABLE	malloc / free	Ownership / Drop
Adatátadás	Tömb, argumentum	Pointer	Borrowing, slice

Kulcs: azonos eredmény ≠ azonos tervezési kompromisszum

Öt szám. Három nyelv. Egy eredmény.

Bemenet: 12 · 18 · 25 · 31 · 44

FORTRAN

```
total = sum(scores)
average = real(total, real64) / &
          real(size(scores), real64)
```

C

```
for (size_t i = 0; i < count; ++i)
  total += scores[i];
average = (double)total / count;
```

RUST

```
let total: i32 = scores.iter().sum();
let average = f64::from(total) /
              scores.len() as f64;
```

SUM: 130 | AVERAGE: 26.00

Ki felel a memória biztonságáért?

A különbség nem csupán a szintaxis.

FORTRAN

Magasabb szintű tömb

`ALLOCATE / DEALLOCATE`

A nyelv kezeli az allocatable tömbök élettartamának több részletét.

C

Közvetlen kontroll

`malloc() / free()`

A programozónak kell gondoskodnia a foglalás és a felszabadítás helyességéről.

RUST

Fordításidejű szabályok

`ownership / borrow`

Sok élettartam- és aliasolási problémát a fordító vizsgál.

A régi nyelvek nem tűntek el

A fejlődés nem lecserélés, hanem újabb lehetőségek hozzáadása.

FORTRAN	**Numerikus és tudományos számítások**	A képletek és tömbök tovább élő világa.
C	**Operációs rendszerek és könyvtárak**	Szintaktikai és interfész-örökség.
RUST	**Biztonságosabb rendszerszoftverek**	Fordítói garanciák és ownership.

A legjobb nyelv mindig a feladattól és a kompromisszumoktól függ.

Három korszak. Három hangsúly.

A programozási nyelvek fejlődése a kompromisszumok újratervezése.

SZÁMÍTÁS

FORTRAN

KONTROLL

C

BIZTONSÁG

RUST

Köszönjük a figyelmet!

Milán · Gergő · Dominik